UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 1 2025/2026

PROJECT 2

SECP 3623 – DATABASE PROGRAMMING

TITLE : EzTransport System

SECTION 01

LECTURER: DR. ROZILAWATI BINTI DOLLAH

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
LUBNA AL HAANI BINTI RADZUAN	A23CS0107
NURAISYAH BINTI MOHD ZIKRE	A23CS0160
HARINI A/P SANGARAN	A23CS0081

Table of Content

1.0 Introduction

This project focuses on a comprehensive transportation booking system inspired by Trip.com platforms. The system will manage the user profile, variety of transportation mode (flights, trains, cars, ferries), booking transactions and payment for the booking made. The system will be able to manage complex schedules and multiple passenger bookings at the same time efficiently.

1.1 Objective

The objective of the chosen system is to to:

- To design and implement a NoSQL backend using MongoDB.
- To perform CRUD operation and implement advanced indexing and aggregation pipeline for each team member.
- To use the embedded documents and references by applying Document Data Model.

1.2 Team Member Role

Each team member is given a task fairly by the leader. The division for the project is shown in **Table 1.**

Name	Task
Nurul Ika Syafiny	● Design and implement TravelOptions collection ● Update Operation ● Sorting ● Security Limitations
Lubna Al Haani	● Design and implement Payment collection ● Query Operations ● Indexing ● Exporting and formatting collections
Nuraisyah	● Design and implement Booking collection ● Introduction ● Delete operation ● Aggregate operation
Harini	● Design and implement Users collection ● Insert operations ● Aggregate operation ● Conclusion

2.0 NoSQL Data Model

Collection Name and Description

User: This collection stores passenger profiles. It uses the Embedded Document pattern to store a preferences object, which contains the user's preferred currency and seat type. This design allows the system to retrieve both identity details and personalized settings in a single read operation. Additionally, the collection tracks account history via the created_at timestamp and operational data such as login_count, supporting a more responsive and customized user experience.

TravelOptions: This is a polymorphic collection that stores all available travel services, including flights, trains, cars, and ferries. Instead of separate tables for each mode of transport, we use a single collection to simplify cross-platform searches and analytics. This collection stores all details about the transportation service offered by the EzTransport system such as its provider, route, base price, amenities, available seats, departure times and its status.

Booking: This collection links both users and travel options. This maintains data integrity by ensuring that updates to a user's profile or a transport schedule automatically reflect in the booking without data redundancy.

Payment: The Payment collection is the final step collection for the EzTransport system. This collection records successful payment including the final amount the customer has paid for the booking and its payment method accordingly. By recording real-time payment information from various payment gateway, this collection allows revenue tracking, passenger payments verifying and also analyzes the effectiveness of codes promotional campaigns.

Example JSON documents

User - The primary key _id is a unique ObjectId automatically generated by MongoDB to ensure each traveler's record is distinct. The collection consists of fields named full_name, email, phone and a timestamp named created_at. The user's personalized settings are stored as an Object within the preferences field, which includes currency and seat type, allowing the system to access both account information and travel preferences in a single read operation. An example of a document in the Users collection can be seen clearly in Figure 1.

Figure 1: Users JSON Documents Example

TravelOptions - The primary key, _id is a unique ObjectId automatically generated by MongoDB. The origin and destination are stored as an Object within the route allowing the system to retrieve all trip details in a single query without needing to join separate tables. Amenities is an array that stores multiple values in a single list, demonstrating MongoDB's ability to handle arrays for multi-valued attributes. An example of a document in the TravelOptions collection can be seen clearly in Figure 2.

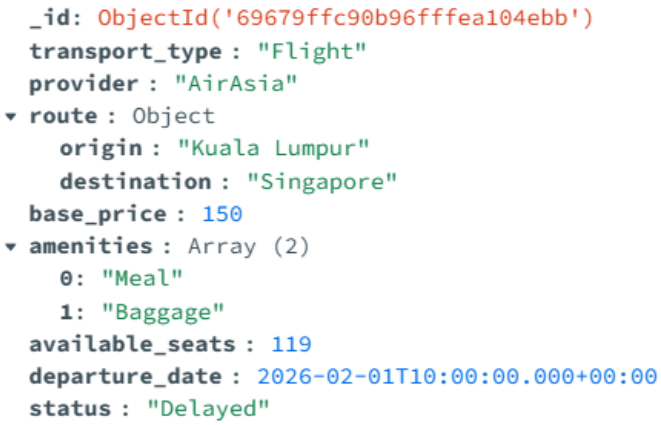

Figure 2: TravelOptions JSON Documents Example

Bookings - The primary key (_id) is automatically generated by MongoDB. The user_id and travel_id fields store the unique identifiers of documents from the Users and TravelOptions collections, respectively where it links travelers to trips. This collection also uses an Array of

Objects for passengers field to store the details of the travelers (name, age, identification) in one document.

Figure 3: Booking JSON Documents Example

Payment - The primary key, _id stored as ObjectId is auto-generated by MongoDB. This collection utilizes document referencing (booking_id) to link each payment to each corresponding booking. The transaction_details field is an embedded document used for storage flexibility in storing the payment gateway metadata such as card details or bank information while promo_codes field is an array to handle multiple applied promo codes in a single transaction. Payment_method, pay_amount, currency, and payment_date are stored as standard scalar fields (String, Double, and Date) to record the essential financial information. A JSON document example of Payment collection can be referred to Figure 4.

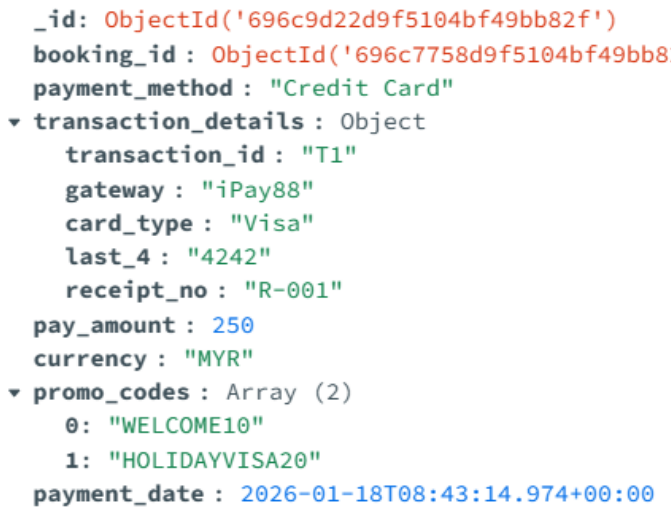

Figure 4: Payment JSON Documents Example

Data Types Used

To ensure data integrity and support advanced operations like aggregation, the following BSON data types are implemented for each collection.

Users:

Field	Data Type	Purpose
_id	ObjectId	Unique identifier for each document.
full_name	String	Stores users' full name.
email	String	Stores users' email details.
phone	String	Stores users' phone number.
preferences	Object	Embedded Document Example: {"currency": "MYR", "seat_type": "Window"}.
created_at	Date	Stores timestamp of account creation.

Table 2: Users Data Types

TravelOptions:

Field	Data Type	Purpose
_id	ObjectId	Unique identifier for each document.
transport_type	String	Categorizes the mode of transport.
base_price	Number (Decimal)	Stores monetary values for calculations.
departure_date	Date	Allows for time-based filtering and sorting.
amenities	Array	Stores a list of features or services.
route	Object	Stores embedded document structure for origin/destination.

Table 3: TravelOption Data Types

Booking:

Field	Data Type	Purpose
_id	ObjectId	Unique identifier for each document.
user_id	ObjectId	Reference to Users collection
travel_id	ObjectId	Reference to TravelOptions collection
passengers	Array	List of travelers that contain their details (name, age, identification)
booking_date	Date	Date of the booking
status	String	Status of booking (Pending/Confirmed/Cancelled)
total_amount	Number	Total price

Table 4: Booking Data Types

Payment:

Field	Data Type	Purpose
_id	ObjectId	Unique identifier for each document.
booking_id	ObjectId	References to Booking collection
payment_method	String	Store the category of the payment (e.g. E-Wallet, Card)
transaction_details	Object	Stores the Gateway-specific metadata
pay_amount	Double/Decimal	Record the final payment amount
promo_codes	Array	Stores list of strings (discount codes)
payment_date	Date	Transaction timestamp

Table 5: Payment Data Types

3.0 CRUD Implementation Summary

3.1 Insert Operations

3.1.1 insertOne

This operation shows how a value for all the fields in a collection is inserted. The query in Figure 5 shows the process of inserting a value in the fields full_name, email, phone, currency, seat_type and created_at in the Users collection.

```
> db.Users.insertOne({
    full_name: "Ahmad Zaki",
    email: "zaki.ahmad@email.com",
    phone: "60123456789",
    preferences: { currency: "MYR", seat_type: "Window" },
    created_at: new Date()
  });
< {
    acknowledged: true,
    insertedId: ObjectId('696e0525d567ddaf1578ea49')
  }
```

Figure 5: insertOne query example

3.1.2 insertMany

This operation shows how multiple entries are added in a collection simultaneously. The query in Figure 6 shows the process of inserting multiple values in the fields full_name, email, phone, currency, seat_type and created_at in the Users collection.

Figure 6: insertMany query example

3.2 Query Operations

3.2.1 findOne

This operation shows how to retrieve a single document that matches a specific criteria which is useful for quick lookups. The query in Figure 7 searches for the first document where the transaction_details.gateway is "iPay88".

Figure 7: findOne query example

3.2.2 find and find $gt, $lt

Unlike findOne, find is used to retrieve many documents that match specific query at once. Condition $gt and $lt is used together with find operations to specify the query. $gt is used for querying the document that is greater than value input while $lt is the opposite which is less than value input. For example, Figure 8 retrieved documents that have pay_amount greater than 100 categorized by currency “MYR”. While Figure 9 retrieved documents that have pay_amount less than 50 and not categorized by any field.

Figure 8 and 9: find and find using $gt and $lt query example

3.2.4 find $eq

Find using $eq is used to perform strict equality matches. Although MongoDB matches equality by default, using the $eq operator formally defines the condition. For example, Figure 10 retrieved documents that have a transaction gateway equal to “FPX” only.

Figure 10: Find using condition $eq query example

3.2.5 find $in

Find using $in is almost similar to $eq but not strictly. It is used to perform equality matches with multiple values within the same field. It will retrieve any document that has matches with any values input. For instance, Figure 11 retrieved documents that have a transaction payment method equal to either E-wallet or Bank Transfer.

Figure 11: Find using condition $in query example

3.2.6 find $and and find $or

The find using $and and $or is almost similar to $in but with a range of multiple fields. However, the difference between $and and $or is $and will retrieve only documents that satisfy all the listed conditions simultaneously while $or will retrieve documents that satisfy any of the listed conditions. For instance, Figure 12 retrieved documents that have a transaction payment_method equal to “Credit Card” and pay_amount greater than 200. On the other hand, Figure 13 retrieved documents that have a pay_amount less than 50 or promo_codes “HOLIDAY_SAVE20”.

Figure 12 and 13: find using $and and $or query example

3.2.8 find for array query

This operation demonstrates how to query inside array structures to find specific combinations of data. The query filters for documents where the promo_codes array contains both "WELCOME10" and "HOLIDAY_SAVE20". This demonstrates the power of the $all operator in identifying complex user behaviors, such as customers who "stack" multiple vouchers in a single transaction. Figure 14 shows the specific transaction that utilized both codes.

```
> db.Payment.find({
    promo_codes: { $all: ["WELCOME10", "HOLIDAY_SAVE20"] }
  });
< {
    _id: ObjectId('696c9d22d9f5104bf49bb82f'),
    booking_id: ObjectId('696c7758d9f5104bf49bb829'),
    payment_method: 'Credit Card',
    transaction_details: {
      transaction_id: 'T1',
      gateway: 'iPay88',
      card_type: 'Visa',
      last_4: '4242'
    },
    pay_amount: 250,
    currency: 'MYR',
    promo_codes: [
      'WELCOME10',
      'HOLIDAY_SAVE20'
    ],
    payment_date: 2026-01-18T08:43:14.974Z
  }
```

Figure 14: find for array query example

3.2.9 find using projection

This operation demonstrates how to return only specific data fields required while excluding others. The query retrieves all payments but uses a projection block to include booking_id, pay_amount, and promo_codes, while excluding the _id field. Figure 15 shows the clean output containing only the requested fields.

```
> db.Payment.find(
    {},
    { booking_id: 1, pay_amount: 1, promo_codes: 1, _id: 0 }
  );
< {
    booking_id: ObjectId('696c7758d9f5104bf49bb829'),
    pay_amount: 250,
    promo_codes: [
      'WELCOME10',
      'HOLIDAY_SAVE20'
    ]
  }
  {
    booking_id: ObjectId('696c7758d9f5104bf49bb82a'),
    pay_amount: 40.5,
```

Figure 15: find using projection query example

3.3 Update Operations

3.3.1 updateOne

In this example, we utilize updateOne() method to demonstrate a targeted update to a specific flight record using both administrative and inventory-based logic.The update targets a document where the provider is “AirAsia” and the route.destination is “Singapore”. The query $set will update its status field to “Delayed” demonstrating the ability to update state information without affecting other fields. Query $inc is used to decrement the available_seats by -1 whenever a booking is confirmed to prevent overbooking. The output in Figure 16 confirms matchedCount: 1 and modifiedCount: 1, indicating the document was successfully found and updated.

Figure 16: updateOne with $set and $inc query example

3.3.2 updateMany $push

This operation will show how to apply bulk updates to a specific category of transport options. The query filters all documents where the transport_type is “Ferry”. The array operator $push adds the new string “Free Water” into the existing amenities array. This demonstrates efficient schema evolution where we can add perks to all ferry options simultaneously. Figure 17 shows a modifiedCount: 3 at the output, confirming that all ferry records in the collection were updated in a single execution.

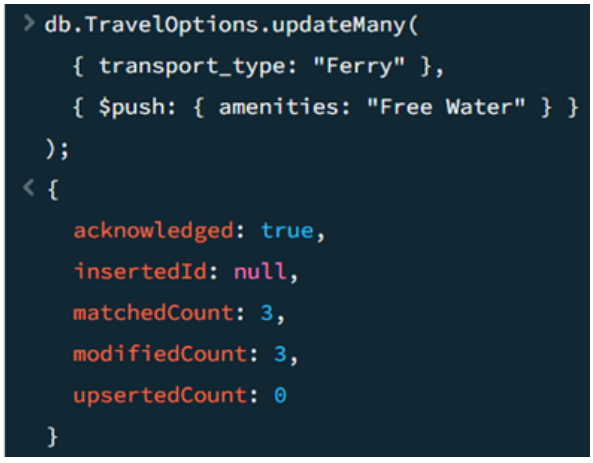

Figure 17: updateMany with $push query example

3.3.3 updateMany $pull

This operation illustrates the removal of specific elements from an array across multiple documents. The filter targets all records categorized as "Car" under the transport_type field. It uses the $pull operator to remove "GPS" from the amenities array. This is useful for inventory management when specific features are no longer available for a certain transport mode. The output in Figure 18 shows matched Count: 2, indicating the query found two car-type documents.

Figure 18: updateMany with $push query example

3.4 Delete Operations

3.4.1 deleteOne

The usage of this operation is to remove a specific record using its unique id, such as when a user cancels a booking. The demonstration of this operation shows its ability to perform a specific data removal from the database without affecting the other documents in the collection.

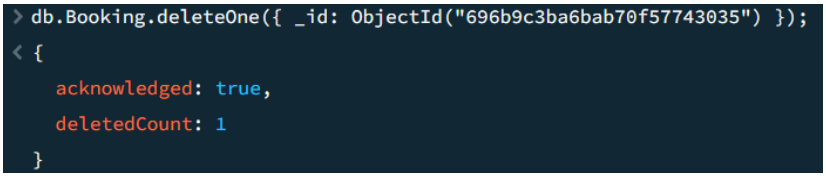

Figure 19: deleteMany query example

3.4.2 deleteMany

This operation is used for administrative clean-up, such as removing all "Cancelled" status bookings from the database to save storage space. This operation demonstrates an operation that deletes a record by filtering the records, this will ensure that the database remains optimized by removing the unnecessary data at once.

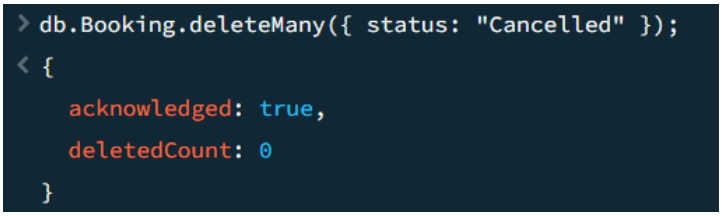

Figure 20: deleteMany query example

4.0 Advanced Operations

4.1 Indexing

The primary function of indexing is to improve the speed of data retrieval operations (queries) in your database. In this project, we used both single and compound indexing.

4.1.1 Single Indexing

In the Figure 21, an index is created on payment_method field where 1 represents ascending order sorting. Indexing said field is important because it is a filter frequently used in a business to keep track of the payment method in a transaction in daily report-making. With indexing, the database has no need to do a complete collection scan, instead, it can easily locate the specific payment method instantly. Figure 21 confirms the index creation with the name payment_method_1.

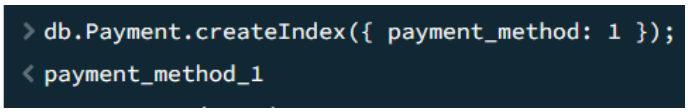

Figure 21: Single indexing creation example

4.1.2 Compound Indexing

Similarly to single index, compound indexing is also implemented by creating indexes on the field but it allows creating indexes on multiple fields in one command. Figure 22 shows an example of compound indexing where indexes are created on transaction_details.gateway in ascending order (1) and also on pay_amount but in descending order (-1). This indexing is created for financial reporting queries that need to filter by a provider (e.g., "iPay88") and immediately find the highest transactions. Figure 22 confirms the index creation with the name transaction_details.gateway_1_pay_amount_–1.

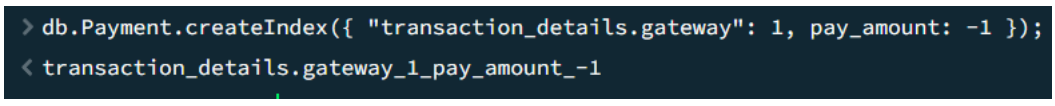

Figure 22: Compound indexing creation example

4.2 Aggregation

For the aggregation pipeline, we used a 3 stages pipeline in Booking collection where in the first stage we used $match operation to filter out the total price that is not greater than zero. The pipeline is continued by grouping ($group operation) the booking by status of the booking to find the total price and the number of bookings made. Lastly, the $sort operation is used to categorize the total price in descending order (-1). The resulting pipeline identified that the “Confirmed” booking status generated a total of 595 in revenue for 3 transactions.

Figure 23: Aggregation pipeline example under Bookings collection

Figure 24 shows another example of an aggregation pipeline which is done under Users collection. The $match operation was used to filter for users who have a specific seat preference, such as "Window". The pipeline then proceeded to the $project stage. By setting _id: 0, the default unique id will not be displayed. Simultaneously, a new field label, _name, was created to map the existing full_name field. This demonstrates the system's ability to rename fields and limit output only to relevant data points for cleaner reporting. $limit is used to manage the output size which is 2 in this case.

Figure 24: Aggregation pipeline example under Users collection

4.3 Sort

In a transportation booking system, providing users with the ability to organize search results such as finding the cheapest travel option or the soonest departure is a critical functional requirement. The following implementation demonstrates how we sort travel data in the TravelOptions collection using the MongoDB sort() method.

4.3.1 Sort Cheapest First

db.TravelOptions.find().sort({ base_price: 1 }); command is used to implement the sort operations. The value 1 indicates Ascending Order which means the documents will be sorted from the cheapest to most expensive. The output in Figure 25 displays the "Langkawi Ferry Line" with a price of 21 as the first result, followed by "KTM ETS" at 45. This confirms the database is correctly sequencing documents from the lowest to the highest price.

Figure 25: Sort Operations Descending Order

4.3.2 Sort Most Expensive First

db.TravelOptions.find().sort({ base_price: -1 }); command is used to implement the sort operations. The value -1 indicates Descending Order which means the documents will be sorted from the most expensive. The output in Figure 26 shows Malaysia Airlines at the top with a base_price of 3200, followed by Star Cruises at 1200. This is useful for users looking for high-end or luxury travel experiences, such as international flights or cruise packages.

Figure 26: Sort Operations Descending Order

4.4 Query Performance Discussion

In MongoDB, indexing is a main mechanism for optimizing query performance in databases. Conceptually, an index functions like the index at the back of a textbook. Without it, the database engine needs to do a collection scan (COLLSCAN) to examine each document in the database to find matches. This process is generally an expensive and inefficient operation as the data grow since it is an O(N) operation which query time grows linearly with data growth.

This is where indexing comes in, when indexing is created, the database engine will create a separate and sorted data structure which is typically a B-Tree structure where it will store the field value and pointer to the actual document. This allows the engine to use index scan (IXSCAN) to traverse through the index tree structure instead of doing full scan which improves query operation time complexity to O(log N) .

5.0 Security & Limitations

As our system transitions from a relational to a NoSQL architecture using MongoDB, we must address specific security vulnerabilities and structural trade-offs inherent in non-relational databases.

Unlike traditional RDBMS, NoSQL systems often prioritize scalability and performance, which can lead to unique security exposures if not properly configured.

- **Access Control**: We implement Role-Based Access Control (RBAC) to ensure that only authorized users like system administrators can perform sensitive operations like deleteMany() or index creation.
- **NoSQL Injection Risks**: Although MongoDB does not use SQL, it is susceptible to "NoSQL Injection". Attackers may attempt to bypass authentication by using operator-based queries like $gt: "" in login fields. To mitigate this, our system must strictly validate and sanitize all user inputs before they reach the query engine.

Operating in a distributed environment requires us to manage the trade-off between Consistency, Availability, and Partition Tolerance (CAP Theorem).

- **Eventual Consistency**: To maintain high availability for booking searches across different regions, our system may favor eventual consistency. This means that a user in one location might see 10 available seats for a flight, while a user in another location might see 9 for a brief moment until the nodes synchronize.
- **Write Concerns**: For critical operations like payment processing in our Payments collection, we enforce a "Majority" write concern to ensure data is confirmed on multiple nodes before returning success, preventing data loss during network partitions.

While MongoDB offers a flexible schema, we must work within certain technical boundaries:

- **Document Size Limit**: Individual documents (such as a single TravelOption) cannot exceed 16MB. While this is sufficient for our current data, extremely long arrays of amenities or logs could eventually hit this limit.
- **Joins (Lookup)**: Unlike SQL, MongoDB does not support traditional JOINs across different collections efficiently. We have minimized this limitation by using Embedding for routes and amenities, which ensures fast read performance without requiring complex cross-collection lookups.

6.0 Teamwork

Nurul Ika Syafiny Binti Azhar

In this project, my primary responsibility was the design and implementation of the TravelOptions collection, which serves as the core inventory for our transportation booking system. I authored the NoSQL data model for this collection, strategically utilizing embedded documents for route details and arrays for amenities to ensure high data locality and minimal

read latency. I was responsible for populating the collection with diverse sample data representing flights, trains, cars, and ferries to test the system's polymorphic capabilities. I also implemented the CRUD operations into the collection for dynamic service update and excellent system performance. I successfully implemented and justified a single-field index on transport types and a compound index on routes to optimize user search speeds. Additionally, I developed aggregation pipelines to provide business insights, such as calculating total available capacity and identifying top-rated travel options, ensuring our backend met both functional and analytical requirements.

Nuraisyah Binti Mohd Zikre

For this project, my task is to design and implement the Booking collection where it links both Users collection and TravelOptions collection by using Reference (ObjectId). To handle the multiple attributes at one time, I utilized the usage of arrays of objects. This way the system can handle multiple travelers per booking. Besides, I also executed the full CRUD implementation for the booking collection. For advanced operation, I used a 3 stages pipeline for aggregation of the system to provide insights for total revenue and the number of bookings.

Lubna Al Haani Binti Radzuan

My part for this project is to design and implement the Payment collection in the EzTransport system. In this collection, I implement document referencing to link each payment to corresponding booking. Other than that, I also utilize embedded documents to store transaction details such as payment method and transaction id in one field for easy categorization. In addition, I am also responsible for query operations and indexing. Query operations have included the use of find() and findOne() coupled with comparison ($gt, $lt) and logical operators ($and, $or) to filter financial data effectively. Finally, I optimized the system performance by implementing both single and compound indexes to ensure query optimization for frequent business queries.

Harini A/P Sangaran

For this project, I designed and implemented the Users collection which is responsible for storing all the information about users of the system. I used embedded documents for the preferences field on the collection where the preferred currency and the preferred seat type were merged, making it easy to retrieve the information for offering personalised experience for the users. Basic CRUD operations and advanced operations such as indexing, aggregation and sorting were also included in the NoSQL codes. Apart from that, I’m responsible for explaining about the insert operations used in the NoSQL codes, which were insertOne() and insertMany(). I also explained about aggregation used under Users collection which used $match, $project and $limit.

7.0 Conclusion

The development of the EzTransportationDB system demonstrates the significant advantages of using a NoSQL Document Data Model over traditional relational structures for modern, multi-modal transportation platforms. The system successfully combines several travel services such as flights, trains, cars, and ferries into a single polymorphic collection, making cross-platform search and data management easier.

The use of Embedded Documents within the Users and Payments collections ensures that high-frequency data, such as traveler preferences and transaction details, are retrieved with minimal latency, optimizing system performance. Furthermore, the implementation of Referencing in the Booking collection maintains strict data integrity while preventing redundancy. The system demonstrates that this NoSQL architecture is scalable and able to meet the real-time, data-intensive demands of the EzTransport ecosystem by using intricate aggregation pipelines.

8.0 Presentation URL

https://youtu.be/a8I138_7flk